

Phase 6 Report

Kubernetes Workload Analyzer — link resource findings to Grafana

Phase status	DONE (2026-09-26)
Report date	2026-09-26
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	5e269f3 , merged as fdaa823 (PR #11)
Backend change	none — client-side only
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 6 was for
2. Why no backend change was needed
3. Confirming the dashboard and its variables via the real Grafana API
4. What links where
5. Security posture kept unchanged
6. Git and GitHub workflow
7. Open items and next phase

1. What Phase 6 was for

The dashboard (Phase 5) shows evidence numbers but not raw historical series beyond the one uPlot chart on Finding Detail. Grafana already has that (D-002: "Grafana remains for raw time-series investigation"). Phase 6's job was narrow: give resource findings and the Workloads page a direct link into Grafana's own bundled dashboard, pre-filtered to the exact workload — not build a second charting surface. Done condition (`PHASES.md`): each resource finding links to its panel.

2. Why no backend change was needed

A Grafana deep link is a URL, not a query result — it can be built entirely from data the frontend already has (namespace, workload name, kind) plus the Grafana base URL. Built entirely client-side in `frontend/src/grafana.ts`, with no backend endpoint added.

3. Confirming the dashboard and its variables via the real Grafana API

kube-prometheus-stack ships a bundled "Kubernetes / Compute Resources / Workload" dashboard. Rather than guessing its UID and template-variable names from memory, both were confirmed against the real running Grafana's API before being hardcoded into `grafana.ts` — the dashboard's UID and its `namespace / type/workload` template variables.

4. What links where

Source	Link target
R001/R002 findings (Finding Detail)	the bundled Workload dashboard, pre-filtered to that exact namespace + workload + type via template variables
Workloads page	same dashboard, filtered per row

Health findings (R003/R004) do not get a Grafana link — a restart/OOM event is a discrete fact already shown in the finding's own evidence, not something better explained by a raw time series panel.

5. Security posture kept unchanged

Grafana's existing login requirement is left as-is — no anonymous-access change was made to enable the link. This matches the threat model's read-only/least-privilege posture (`docs/threat-model.md`: "Unauthorized dashboard access... Auth is required before any non-local exposure"); the link simply opens Grafana's normal login-gated URL in a new tab.

6. Git and GitHub workflow

Branch `phase-6` → commit `5e269f3` ("feat(phase-6): link resource findings to Grafana") → PR #11 → merged as `fdaa823` → branch deleted.

7. Open items and next phase

- No in-cluster deployment yet — Phase 7.
- No scripted demo runbook yet — Phase 8.